

Open Notebook Production Guide - Chapters 2-8

Chapter 2: Model Strategy — Choosing and Optimizing Your AI Stack

Estimated reading time: 18 minutes **What you'll build:** A complete model configuration optimized for your budget and use case, with per-task model assignments, cost projections, and Ollama tuning. **Prerequisites:** Chapter 1 — working Docker deployment with at least one provider configured.

Open Notebook's killer feature isn't any single model. It's that you can route different tasks through different models. This chapter picks the right models for each task, at your budget.

The Three Tasks and What They Need

Open Notebook does three fundamentally different AI tasks. Each needs a different kind of model:

Task	What it does	What matters	Ideal model traits
Chat / Analysis	Answers questions about your documents, generates summaries, writes notes	Reasoning quality, context length, factual accuracy	Large, smart models

Task	What it does	What matters	Ideal model traits
Embedding	Converts document text into vectors for semantic search	Retrieval accuracy, speed, batch throughput	Specialized embedding models
Podcast TTS	Generates spoken audio for multi-speaker podcasts	Voice quality, naturalness, speaker consistency	TTS-only models (not LLMs)

Using a \$15/million-token model for embeddings is burning money. Using a \$0.02/million-token model for complex document analysis gives garbage answers. Route intentionally.

Chat/Analysis Models — Picking Your Brain

Cloud Options (No GPU Required)

Model	Provider	Cost per 1M input tokens	Cost per 1M output tokens	Best for
Claude 3.5 Sonnet	Anthropic	\$3.00	\$15.00	Deep analysis, long docs, complex reasoning
GPT-4o	OpenAI	\$2.50	\$10.00	Best general purpose, strong at everything
GPT-4o-mini	OpenAI	\$0.15	\$0.60	Budget option, surprisingly good for summaries
Gemini 2.0 Flash	Google	\$0.10	\$0.40	Best price/performance, free tier robust
Llama 3.3 70B	Groq	\$0.59	\$0.79	Fast, open-source quality, 15K RPM free tier

Model	Provider	Cost per 1M input tokens	Cost per 1M output tokens	Best for
DeepSeek V3	DeepSeek	\$0.27	\$1.10	Strong coding/reasoning, very cheap input

Local Options (GPU Required)

Model	VRAM needed	Quality equivalent	Best for
Llama 3.2 3B	4 GB	GPT-3.5-ish	Quick summaries, testing
Llama 3.1 8B	6 GB	Between GPT-3.5 and 4	Good all-rounder for 12 GB
GPT-OSS 20B	8-10 GB	Close to GPT-4o-mini	Best local quality for 12-16 GB
Qwen 2.5 32B	16-20 GB	Approaching GPT-4o	Best local quality for 24 GB
Llama 3.3 70B	40+ GB	GPT-4o class	Dual GPU or Mac Studio

My Recommended Chat Configs

- **\$0/month (RTX 3060 12GB):** Primary: `gpt-oss:20b` (fits with `num_ctx=4096`).
Fallback: `llama3.2:3b` .
- **\$15/month:** Groq `llama-3.3-70b` (free tier). Heavy queries: GPT-4o-mini.
- **\$50/month:** GPT-4o for analysis. GPT-4o-mini for summaries. Ollama for sensitive docs.
- **\$150/month:** Claude 3.5 Sonnet for analysis, GPT-4o-mini for summaries, Ollama `qwen2.5:32b` on RTX 4090.

Embedding Models — The Engine Nobody Talks About

Bad embedding = right documents not found = AI gives wrong answers even with excellent chat model.

Cloud Embedding

Model	Provider	Cost per 1M tokens	Dimensions	Notes
text-embedding-3-small	OpenAI	\$0.02	512/1536	Default recommendation
text-embedding-3-large	OpenAI	\$0.13	256/1024/3072	When quality matters
voyage-3-lite	Voyage AI	\$0.02	512	Good with technical text
voyage-3	Voyage AI	\$0.06	1024	Better for code/tech

Cost reality: 1000 PDFs (~50M tokens) ≈ \$1.00 one-time.

Local Embedding (Ollama)

Model	VRAM	Quality
nomic-embed-text	1 GB	Good
mxbai-embed-large	2 GB	Better for tech/scientific
bge-m3	3 GB	Multilingual, highest quality

WARNING: Esperanto library bug (Issue #655): Ollama embeddings may fail with "Connection error" even when chat works. Workaround: use OpenAI `text-embedding-3-small` (\$0.02/1M) and keep Ollama for chat only.

Podcast TTS Models

ElevenLabs Model	Cost per 1K chars	Quality
Multilingual v2	~\$0.30	Professional
Turbo v2.5	~\$0.05	Very good (sweet spot)
Flash v2.5	~\$0.01	Acceptable, fast

15-min podcast (~20K chars): Multilingual ≈ \$6.00, Turbo ≈ \$1.00, Flash ≈ \$0.20.

The Model Routing Pattern

Chat (analysis)	→ GPT-4o or Claude	(expensive, high quality)
Title generation	→ GPT-4o-mini	(cheap, simple)
Summary generation	→ GPT-4o-mini	(cheap, frequent)
Embedding	→ text-embed-3-small	(cheapest, specialized)
Podcast TTS	→ ElevenLabs Turbo	(pay per podcast)
Sensitive chat	→ Ollama gpt-oss:20b	(local, zero-cost)

Cost Projections

- **Solo:** ~\$4/month (Groq free + ElevenLabs Turbo)
- **Small team** (500 queries, 200 docs): ~\$16/month (GPT-4o-mini + Turbo + \$5 VPS)
- **Research team** (2000 queries, 1000 docs): ~\$90/month (GPT-4o + mini + Turbo + GPU server)

Ollama Optimization

```
# 1. Context window by VRAM (num_ctx in Settings):
# 8 GB → 2048-4096 | 12 GB → 4096-8192
# 16 GB → 8192-16384 | 24 GB → 16384-32768

# 2. Check loaded models and VRAM:
curl http://localhost:11434/api/ps

# 3. Unload unused models:
curl http://localhost:11434/api/generate -d '{
  "model": "gpt-oss:20b", "keep_alive": 0
}'
```

```
# 4. Multi-GPU:
environment:
  - CUDA_VISIBLE_DEVICES=0,1
```

Provider Gotchas

- **Groq:** 15K RPM free tier but 30 RPM rate limit. Chat only, NOT batch embedding.
- **Anthropic:** Rate limits per-org, not per-key. Teams need separate keys.
- **OpenAI:** Tier 1 = 500 RPM. Batch embeddings 50 at a time.
- **ElevenLabs:** Free tier = 10K chars (~7 min podcast). Starter \$5/mo = 30K. Creator \$22/mo = 100K.
- **DeepSeek:** Short 8K context. Unsuitable for long-document chat.

Next: Chapter 3 — Reverse Proxy, TLS, and Authentication → # Chapter 3: Reverse Proxy, TLS, and Authentication

Estimated reading time: 15 minutes **What you'll build:** A production-grade nginx reverse proxy with Let's Encrypt TLS, firewall hardening, and multi-layer authentication protecting your Open Notebook instance. **Prerequisites:** Chapter 1 — working local deployment. A domain name pointing to your server's IP.

Right now your Open Notebook runs on `http://localhost:8502`. It's accessible only from your machine. This chapter makes it accessible from the internet — securely.

WARNING: Do not skip TLS. Exposing Open Notebook over plain HTTP means API keys, document contents, and chat history travel in cleartext. Anyone on the same network can sniff them.

Complete nginx Configuration

Install nginx and certbot:

```
sudo apt update && sudo apt install -y nginx certbot python3-certbot-nginx
```

Create `/etc/nginx/sites-available/open-notebook` :

```
# — HTTP → HTTPS Redirect —
server {
    listen 80;
    server_name notebook.yourdomain.com;
    location /.well-known/acme-challenge/ {
        root /var/www/html;
    }
    location / {
        return 301 https://$host$request_uri;
    }
}

# — HTTPS Main Server —
server {
    listen 443 ssl http2;
    server_name notebook.yourdomain.com;

    ssl_certificate      /etc/letsencrypt/live/notebook.yourdomain.com/
fullchain.pem;
    ssl_certificate_key  /etc/letsencrypt/live/notebook.yourdomain.com/
privkey.pem;
    ssl_protocols        TLSv1.2 TLSv1.3;
    ssl_ciphers          ECDHE-ECDSA-AES128-GCM-SHA256:ECDHE-RSA-AES128-GCM-
SHA256;
    ssl_prefer_server_ciphers on;
    ssl_session_cache    shared:SSL:10m;
    ssl_session_timeout  10m;

    add_header X-Frame-Options "SAMEORIGIN" always;
    add_header X-Content-Type-Options "nosniff" always;
    add_header X-XSS-Protection "1; mode=block" always;
    add_header Referrer-Policy "strict-origin-when-cross-origin" always;

    client_max_body_size 500M; # For large PDFs

# — Web UI with WebSocket support —
    location / {
        proxy_pass http://127.0.0.1:8502;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;           # WebSocket
        proxy_set_header Connection "upgrade";             # WebSocket
        proxy_read_timeout 86400s;
    }

# — REST API —
```

```

location /api/ {
    rewrite ^/api/(.*) /$1 break;
    proxy_pass http://127.0.0.1:5055;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_http_version 1.1;
    proxy_read_timeout 300s;
}

# — Health check (no auth) —
location /health {
    proxy_pass http://127.0.0.1:5055/health;
    access_log off;
}
}

```

Enable site, get TLS:

```

sudo ln -s /etc/nginx/sites-available/open-notebook /etc/nginx/sites-enabled/

sudo rm /etc/nginx/sites-enabled/default
sudo nginx -t && sudo systemctl reload nginx
sudo certbot --nginx -d notebook.yourdomain.com
sudo certbot renew --dry-run # Verify auto-renewal

```

Update docker-compose.yml:

```

environment:
  - API_URL=https://notebook.yourdomain.com

```

Firewall Hardening

```

sudo apt install -y ufw
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow ssh
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp
sudo ufw enable

```


WARNING: Do NOT open ports 8502, 5055, 8000, or 11434. Users access through nginx on 443 only.

Authentication — Three Layers

Layer 1: Instance Password

```
environment:  
  - OPEN_NOTEBOOK_PASSWORD=your-strong-password
```

Layer 2: nginx Basic Auth

```
auth_basic "Open Notebook — Authentication Required";  
auth_basic_user_file /etc/nginx/.htpasswd;
```

```
sudo apt install -y apache2-utils  
sudo htpasswd -c /etc/nginx/.htpasswd alice  
sudo htpasswd /etc/nginx/.htpasswd bob  
sudo systemctl reload nginx
```

Layer 3: OAuth2 Proxy (Google/GitHub)

```
sudo apt install -y oauth2-proxy
```

/etc/oauth2-proxy/oauth2-proxy.cfg :

```
provider = "google"  
email_domains = ["yourcompany.com"]  
upstreams = ["http://127.0.0.1:8502"]
```

Then point nginx to `http://127.0.0.1:4180` instead of `:8502` .

Cloudflare Access (easiest for Cloudflare users): Zero software install. Add application in Cloudflare Zero Trust, restrict by email domain. Free for ≤50 users.

Rate Limiting

```
limit_req_zone $binary_remote_addr zone=api_limit:10m rate=10r/s;
limit_req_zone $binary_remote_addr zone=web_limit:10m rate=30r/s;

# In location /api/: limit_req zone=api_limit burst=20 nodelay;
# In location /:    limit_req zone=web_limit burst=50 nodelay;
```

fail2ban Brute-Force Protection

```
sudo apt install -y fail2ban
sudo tee /etc/fail2ban/filter.d/open-notebook.conf << 'EOF'
[Definition]
failregex = ^<HOST> -.*"(GET|POST).*/api/.*" 401
           ^<HOST> -.*"(GET|POST).*/api/.*" 403
ignoreregex =
EOF
sudo tee /etc/fail2ban/jail.d/open-notebook.conf << 'EOF'
[open-notebook]
enabled = true
filter = open-notebook
logpath = /var/log/nginx/access.log
maxretry = 10
bantime = 3600
findtime = 600
EOF
sudo systemctl restart fail2ban
```

Security Verification

```
curl -I https://notebook.yourdomain.com | grep -E "HTTP|Strict-Transport"

curl -I http://notebook.yourdomain.com | grep "301"
nmap -p 8502,5055,8000,11434 your-server-ip # Should all be filtered/closed

curl -I https://notebook.yourdomain.com | grep -E "X-Frame|X-Content|X-XSS"
```

Your Open Notebook is now accessible from anywhere, with valid TLS, firewall, authentication, rate limiting, and brute-force protection. In Chapter 4, we scale for production workloads.

Next: Chapter 4 — Production Deployment Patterns → # Chapter 4: Production Deployment Patterns

Estimated reading time: 18 minutes **What you'll build:** Production-ready Open Notebook deployments on a single machine, a split Ollama+App architecture, Kubernetes manifests, and cloud-specific recipes for AWS, GCP, and Hetzner.

Prerequisites: Chapters 1-3 — working local deployment with nginx + TLS.

Your Open Notebook works on localhost. This chapter gets it running like infrastructure — auto-restarting, surviving reboots, scaling across machines, and deployable through CI/CD.

Pattern 1: Single Docker Host (systemd)

```
# /etc/systemd/system/open-notebook.service
[Unit]
Description=Open Notebook Docker Compose Stack
Requires=docker.service
After=docker.service network-online.target

[Service]
Type=oneshot
RemainAfterExit=yes
WorkingDirectory=/opt/open-notebook
ExecStart=/usr/bin/docker compose up -d
ExecStop=/usr/bin/docker compose down
ExecReload=/usr/bin/docker compose up -d --force-recreate
StandardOutput=journal
StandardError=journal

[Install]
WantedBy=multi-user.target
```

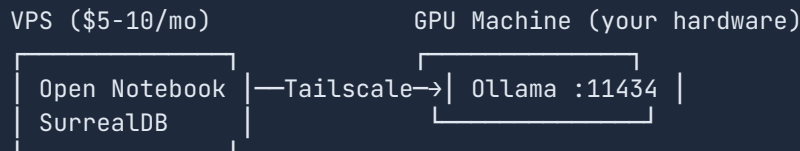
```
sudo systemctl daemon-reload
sudo systemctl enable open-notebook --now
```

Add resource limits and log rotation to docker-compose.yml:

```
services:
  open-notebook:
    deploy:
      resources:
```

```
limits: { cpus: '2.0', memory: 2G }
reservations: { cpus: '0.5', memory: 512M }
logging:
  driver: "json-file"
  options: { max-size: "50m", max-file: "3" }
```

Pattern 2: Split Architecture — Ollama on Separate GPU



Install Tailscale on both machines, then configure:

```
# VPS docker-compose.yml - NO ollama service
environment:
  - OLLAMA_BASE_URL=http://gpu-machine.tailnet.ts.net:11434
```

On GPU machine:

```
curl -fsSL https://ollama.com/install.sh | sh
ollama pull gpt-oss:20b
echo 'Environment="OLLAMA_HOST=0.0.0.0:11434"' | \
  sudo tee /etc/systemd/system/ollama.service.d/override.conf
sudo systemctl daemon-reload && sudo systemctl restart ollama
```

Pattern 3: Kubernetes

For Kubernetes deployments, adapt the official manifests from the Open Notebook GitHub repository to your cluster. Key considerations:

- **Replicas:** 1 (stateful — SurrealDB does not support multi-writer)
- **PersistentVolumeClaim:** For `surreal-data` and `app-data` volumes
- **cert-manager:** For automatic TLS certificate management
- **Ingress:** Route `/api/` to port 5055, everything else to port 8502

See `lfnovo/open-notebook` on GitHub for the latest K8s manifest examples.

Cloud Recipes

Provider	CPU-only	GPU	Monthly
Hetzner CX22	2 vCPU, 4 GB, 40 GB	—	\$4.50
Hetzner CPX31	4 vCPU, 8 GB, 160 GB	—	\$14.00
Hetzner Auction	—	RTX 3060	~\$50
Hetzner Auction	—	RTX 4090	~\$200
AWS g4dn.xlarge	4 vCPU, 16 GB	T4 16 GB	~\$380
AWS t3.medium	2 vCPU, 4 GB	—	~\$30
GCP g2-standard-4	4 vCPU, 16 GB	L4 24 GB	~\$500

CI/CD (GitHub Actions)

```
# .github/workflows/deploy.yml
on:
  push:
    branches: [main]
    paths: ['docker-compose.yml', 'nginx/**']
jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Deploy
        uses: appleboy/ssh-action@v1
        with:
          host: ${ secrets.SSH_HOST }
          script: |
            cd /opt/open-notebook
            docker compose exec -T surrealdb surreal export \
              --conn http://localhost:8000 --user root --pass root \
              --ns open_notebook --db open_notebook \
              > /backups/pre-deploy-$(date +%Y%m%d-%H%M).surql
            docker compose pull
            docker compose up -d --force-recreate
            sleep 15
            curl -sf http://localhost:5055/health || exit 1
```

Health Check Cron

```
# /etc/cron.d/open-notebook-health
*/5 * * * * root curl -sf http://localhost:5055/health || \
echo "ALERT: Open Notebook Down at $(date)" | \
mail -s "ALERT" admin@yourdomain.com
```

Next: Chapter 5 — Data Management and Backup → # Chapter 5: Data Management and Backup

Estimated reading time: 14 minutes **What you'll build:** Automated daily backups with off-site storage, a tested disaster recovery procedure, migration scripts, and data pruning strategies. **Prerequisites:** Chapter 1 — working deployment.

Open Notebook's data lives in three places. You need to back up two of them.

Path	Content	Backup?
./surreal-data/	All notebooks, notes, chat history, embeddings	CRITICAL: MUST
./app-data/	Raw uploaded files, processed chunks	HIGH: SHOULD
./app-config/	UI preferences	OPTIONAL: OPTIONAL
./ollama-data/	Model weights (huge, reproducible)	SKIP: SKIP

Automated Backup Script

Create `/opt/open-notebook/backup.sh` :

```
#!/bin/bash
set -euo pipefail
BACKUP_ROOT="/backups/open-notebook"
TIMESTAMP=$(date +%Y%m%d-%H%M)
```

```

BACKUP_DIR="${BACKUP_ROOT}/${TIMESTAMP}"
RETENTION_DAYS=30

mkdir -p "$BACKUP_DIR"
cd /opt/open-notebook

# Stop app for consistent DB snapshot (brief downtime)
docker compose stop open-notebook
sleep 2

# Export SurrealDB
docker compose exec -T surrealdb surreal export \
    --conn http://localhost:8000 --user root --pass root \
    --ns open_notebook --db open_notebook \
    > "${BACKUP_DIR}/database-export.surql"

# Verify export
if grep -q "DEFINE TABLE" "${BACKUP_DIR}/database-export.surql"; then
    RECORD_COUNT=$(grep -c "CREATE" "${BACKUP_DIR}/database-export.surql" || echo "0")
    echo "Export valid: ${RECORD_COUNT} records"
else
    echo "ERROR: Export failed" && docker compose start open-notebook && exit 1
fi

# Copy app data
tar -czf "${BACKUP_DIR}/app-data.tar.gz" ./app-data/ 2>/dev/null || true

# Restart
docker compose start open-notebook

# Compress
ARCHIVE_NAME="open-notebook-backup-${TIMESTAMP}.tar.gz"
tar -czf "${BACKUP_ROOT}/${ARCHIVE_NAME}" -C "$BACKUP_DIR" .

# Optional: Encrypt with GPG
if command -v gpg &> /dev/null && [ -n "${GPG_RECIPIENT:-}" ]; then
    gpg --encrypt --recipient "$GPG_RECIPIENT" \
        --output "${BACKUP_ROOT}/${ARCHIVE_NAME}.gpg" \
        "${BACKUP_ROOT}/${ARCHIVE_NAME}"
    mv "${BACKUP_ROOT}/${ARCHIVE_NAME}.gpg" "${BACKUP_ROOT}/${ARCHIVE_NAME}"
fi

# Upload off-site (rclone)
if command -v rclone &> /dev/null && [ -n "${RCLONE_REMOTE:-}" ]; then
    rclone copy "${BACKUP_ROOT}/${ARCHIVE_NAME}" "${RCLONE_REMOTE}:/open-notebook-backups/"
fi

# Clean old backups
find "$BACKUP_ROOT" -name "open-notebook-backup-*.tar.gz*" -mtime +${RETENTION_DAYS} -delete
find "$BACKUP_ROOT" -maxdepth 1 -type d -name "202*" -mtime +${RETENTION_DAYS} -exec rm -rf {} \;

```

```
FINAL_SIZE=$(du -h "${BACKUP_ROOT}/${ARCHIVE_NAME}" | cut -f1)
echo "Backup complete: ${ARCHIVE_NAME} (${FINAL_SIZE})"
rm -rf "$BACKUP_DIR"
```

Schedule:

```
chmod +x /opt/open-notebook/backup.sh
echo "0 3 * * * root /opt/open-notebook/backup.sh" | \
    sudo tee /etc/cron.d/open-notebook-backup
```

Note: The backup script stops the app for ~5-10 seconds to get a consistent database snapshot. Schedule it during low-usage hours.

Off-Site Storage (rclone)

- **Backblaze B2:** `rclone config` → b2 → 10 GB free tier (~3 GB for 30 daily backups)
- **AWS S3:** `rclone config` → s3 → Standard tier
- **Google Drive:** `rclone config` → drive → 15 GB free

Set `export RCLONE_REMOTE="b2"` (or s3/drive) in the backup script.

Disaster Recovery: Full Restore

```
# Fresh server with Docker installed
cd /opt/open-notebook
tar -xzf /path/to/backup.tar.gz -C /tmp/restore/

# Fresh deploy
curl -O https://raw.githubusercontent.com/lfnovo/open-notebook/main/docker-
compose.yml
sed -i 's/change-me-to-a-secret-string/YOUR-ORIGINAL-SECRET-KEY/' docker-
compose.yml
docker compose up -d surrealdb
sleep 10

# Restore database
cat /tmp/restore/database-export.surql | docker compose exec -T surrealdb \

    surreal import --conn http://localhost:8000 \
        --user root --pass root --ns open_notebook --db open_notebook

# Restore files
```



```
tar -xzf /tmp/restore/app-data.tar.gz -C ./

# Start and verify
docker compose up -d
curl http://localhost:5055/health
```

Test your restore procedure monthly. [An untested backup is a prayer.](#)

Migration: Moving Servers

```
# Old server
docker compose stop
tar -czf /tmp/migration.tar.gz ./surreal-data/ ./app-data/ ./app-config/
docker-compose.yml
scp /tmp/migration.tar.gz user@new-server:/tmp/

# New server
cd /opt/open-notebook && tar -xzf /tmp/migration.tar.gz && docker compose up -d
```

Data Pruning & Storage Growth

Usage	Monthly growth
Light (1-2 users)	1-2 GB
Medium (5 users)	5-10 GB
Heavy (20+ users)	30-50 GB

```
-- Prune old sources
DELETE FROM source WHERE created_at < time::now() - 6w;
```

Export old notebooks to JSON before deleting for cold storage.

Monthly Maintenance Script

```
#!/bin/bash
docker compose exec surrealdb surreal sql --conn http://localhost:8000 \
```

```
--user root --pass root --ns open_notebook --db open_notebook \  
--query "  
SELECT 'notebooks', count() FROM notebook GROUP ALL;  
SELECT 'sources', count() FROM source GROUP ALL;  
"  
du -sh /opt/open-notebook/surreal-data/ /opt/open-notebook/app-data/
```

Next: Chapter 6 — Multi-User and Team Setup → # Chapter 6: Multi-User and Team Setup

Estimated reading time: 13 minutes **What you'll build:** A multi-user Open Notebook instance with workspace isolation, usage monitoring, cost allocation per user, and admin controls. **Prerequisites:** Chapters 1-3 — working deployment with authentication.

Open Notebook v1.x treats all users as equal citizens. No built-in RBAC. This chapter layers isolation, monitoring, and cost tracking on top.

Pattern 1: Separate Instances Per Team

Run multiple stacks on different ports:

```
/opt/open-notebook/team-alpha/ → :8502  
/opt/open-notebook/team-beta/ → :8503  
/opt/open-notebook/team-legal/ → :8504 # Ollama-only, zero cloud
```

Each team uses their own API keys → billing goes to their department. nginx routes by subdomain. Complete isolation, simple cost attribution.

Pattern 2: OAuth2 Proxy with Per-User Identity

```
location / {  
    auth_request /oauth2/auth;  
    error_page 401 = /oauth2/sign_in;  
    auth_request_set $user $upstream_http_x_auth_request_user;  
    proxy_set_header X-Forwarded-User $user;  
    proxy_set_header X-Forwarded-Email $upstream_http_x_auth_request_email;
```

```
    proxy_pass http://127.0.0.1:8502;
}
```

User logs in via Google → email becomes user identifier → each person sees only their notebooks.

Cost Allocation Per User

```
# Extract per-user API usage from nginx logs
grep "$(date +%d/%b/%Y)" /var/log/nginx/access.log | \
    grep "/api/" | awk '{print $1}' | sort | uniq -c | sort -rn

# With OAuth2 Proxy:
grep "X-Forwarded-Email" /var/log/nginx/access.log | \
    awk -F'X-Forwarded-Email: ' '{print $2}' | awk '{print $1}' | \
    sort | uniq -c | sort -rn
```

Resource Allocation

Limit Ollama concurrency:

```
environment:
  - OLLAMA_NUM_PARALLEL=2
  - OLLAMA_MAX_LOADED_MODELS=1
```

Split GPU: chat vs podcast on separate GPUs:

```
ollama-chat:
  environment: { CUDA_VISIBLE_DEVICES: 0 }
  ports: ["11434:11434"]
ollama-podcast:
  environment: { CUDA_VISIBLE_DEVICES: 1 }
  ports: ["11435:11434"]
```

Rate limit podcasts in nginx:

```
location /api/podcast/ {
    limit_req zone=podcast_limit burst=1 nodelay;
```

```
proxy_pass http://127.0.0.1:5055;
}
```

Usage Report Script

```
#!/bin/bash
echo "=== Usage Report - $(date) ==="
docker compose exec -T surrealdb surreal sql --conn http://localhost:8000 \

    --user root --pass root --ns open_notebook --db open_notebook \
    --query "
    SELECT count() AS notebooks FROM notebook GROUP ALL;
    SELECT count() AS sources FROM source GROUP ALL;
    SELECT type, count() FROM source WHERE created_at > time::now() - 1d GROUP
BY type;
    "
du -sh /opt/open-notebook/surreal-data/ /opt/open-notebook/app-data/
nvidia-smi --query-gpu=utilization.gpu,memory.used,memory.total --format=csv,n
oheader 2>/dev/null
```

Offboarding

```
# Export first, then delete
docker compose exec surrealdb surreal sql ... \
    --query "SELECT * FROM notebook WHERE user = 'departing@company.com'" \

    > /exports/departing-$(date +%Y%m%d).json

# Delete their data
docker compose exec surrealdb surreal sql ... \
    --query "
    DELETE FROM note WHERE notebook.user = 'departing@company.com';
    DELETE FROM source WHERE notebook.user = 'departing@company.com';
    DELETE FROM notebook WHERE user = 'departing@company.com';
    "

sudo htpasswd -D /etc/nginx/.htpasswd departing@company.com
sudo systemctl reload nginx
```

Alert Thresholds

Metric	Warning	Critical
Disk usage	>70%	>85%
GPU memory	>80%	>95%
API error rate	>5%	>10%
Backup age	>25 hrs	>30 hrs
Health check	1 fail	3 consecutive

Next: Chapter 7 — API-Driven Workflows and Automation → # Chapter 7: API-Driven Workflows and Automation

Estimated reading time: 16 minutes **What you'll build:** Full API automation — batch document ingestion, automated podcast generation, Python client patterns, webhook integration, and n8n/Zapier recipes. **Prerequisites:** Chapter 1 — working deployment with API port 5055 accessible.

Open Notebook has a full REST API on port 5055. Everything you can do in the UI, you can do programmatically. This chapter turns your instance from a web app into an automation engine.

Pain Point #7: One of the main reasons to self-host instead of using NotebookLM is the API. NotebookLM has no API. Open Notebook's API enables batch processing thousands of documents, building custom frontends, and integrating document AI into existing workflows.

API Reference (Concise)

- **Base URL:** `https://notebook.yourdomain.com/api`
- **Auth:** `Authorization: Bearer your-secret-key` or `X-API-Key: your-secret-key`

Health Check

```
curl -H "Authorization: Bearer $API_KEY" https://notebook.yourdomain.com/api/health
```

Create Notebook

```
curl -X POST https://notebook.yourdomain.com/api/notebooks \
  -H "Authorization: Bearer $API_KEY" \
  -H "Content-Type: application/json" \
  -d '{"title": "Q3 Research", "description": "Market analysis"}'
```

Upload PDF

```
curl -X POST https://notebook.yourdomain.com/api/sources \
  -H "Authorization: Bearer $API_KEY" \
  -F "file=@document.pdf" -F "notebook_id={notebook_id}"
```

Add URL/YouTube

```
curl -X POST https://notebook.yourdomain.com/api/sources \
  -H "Authorization: Bearer $API_KEY" \
  -H "Content-Type: application/json" \
  -d '{"url": "https://example.com/article", "notebook_id": "{id}"}'
```

Chat

```
curl -X POST https://notebook.yourdomain.com/api/chat \
  -H "Authorization: Bearer $API_KEY" \
  -H "Content-Type: application/json" \
  -d '{"notebook_id": "{id}", "message": "Summarize key findings", "stream": false}'
```

Generate Podcast

```
curl -X POST https://notebook.yourdomain.com/api/podcast/generate \
-H "Authorization: Bearer $API_KEY" \
-d '{"notebook_id": "{id}", "speaker_count": 2, "style": "conversational"}'
```

Download Podcast

```
curl https://notebook.yourdomain.com/api/podcast/download/{podcast_id} \
-H "Authorization: Bearer $API_KEY" -o podcast.mp3
```

Python Client: Batch Document Ingestion

```
#!/usr/bin/env python3
import os, time, requests
from pathlib import Path

API_URL = "https://notebook.yourdomain.com/api"
API_KEY = os.environ["OPEN_NOTEBOOK_API_KEY"]
NOTEBOOK_ID = "your-notebook-id"
PDF_DIR = "/path/to/pdfs"

def upload_pdf(filepath):
    with open(filepath, "rb") as f:
        resp = requests.post(f"{API_URL}/sources",
            headers={"Authorization": f"Bearer {API_KEY}"},
            files={"file": (os.path.basename(filepath), f, "application/pdf")},
            data={"notebook_id": NOTEBOOK_ID})
    return resp.json()["id"]

def wait_for_processing(source_id, timeout=300):
    start = time.time()
    while time.time() - start < timeout:
        status = requests.get(f"{API_URL}/sources/{source_id}",
            headers={"Authorization": f"Bearer {API_KEY}"})
        status = status.json().get("status")
        if status == "completed": return "completed"
        if status == "error": return "error"
        time.sleep(5)
    return "timeout"

pdf_files = list(Path(PDF_DIR).glob("*.pdf"))
for i, pdf in enumerate(pdf_files):
    print(f"[{i+1}/{len(pdf_files)}] {pdf.name}...", end=" ")
    source_id = upload_pdf(str(pdf))
```

```

status = wait_for_processing(source_id)
print("[OK]" if status == "completed" else f"[FAIL] {status}")
time.sleep(5)

```

Python Client: Automated Podcast Pipeline

```

import os, time, requests
from datetime import datetime

API_URL = "https://notebook.yourdomain.com/api"
API_KEY = os.environ["OPEN_NOTEBOOK_API_KEY"]

def get_source_count(notebook_id):
    resp = requests.get(f"{API_URL}/sources?notebook_id={notebook_id}",
                        headers={"Authorization": f"Bearer {API_KEY}"})
    return len(resp.json().get("sources", []))

def generate_podcast(notebook_id):
    return requests.post(f"{API_URL}/podcast/generate",
                        headers={"Authorization": f"Bearer {API_KEY}"},
                        json={"notebook_id": notebook_id, "speaker_count": 2}).json()["podcast_id"]

def wait_for_podcast(podcast_id, timeout=600):
    start = time.time()
    while time.time() - start < timeout:
        resp = requests.get(f"{API_URL}/podcast/status/{podcast_id}",
                            headers={"Authorization": f"Bearer {API_KEY}"})
        status = resp.json().get("status")
        if status == "completed":
            return resp.json()["download_url"]
        time.sleep(10)
    raise TimeoutError()

def download(url, path):
    with requests.get(url, stream=True) as r, open(path, "wb") as f:
        for chunk in r.iter_content(8192): f.write(chunk)

if get_source_count("notebook-id") ≥ 5:
    pid = generate_podcast("notebook-id")
    url = wait_for_podcast(pid)
    download(url, f"podcast-{datetime.now():%Y%m%d-%H%M}.mp3")

```

Minimal Custom Frontend (Flask)

```

from flask import Flask, request, render_template_string
import requests, os

```



```

API_URL = "https://notebook.yourdomain.com/api"
API_KEY = os.environ["OPEN_NOTEBOOK_API_KEY"]

HTML = """<!DOCTYPE html><html><head><title>DocQ</title></head>
<body style="max-width:700px;margin:40px auto;font-family:system-ui">
<h1>DocQ</h1>
<form method="post">
    <textarea name="query" rows="3" style="width:100%">{{query}}</textarea>

    <button type="submit">Ask</button>
</form>
{% if answer %}<div style="margin-top:20px;background:#f5f5f5;padding:
15px">{{answer}}</div>{% endif %}
</body></html>"""

app = Flask(__name__)

@app.route("/", methods=["GET", "POST"])
def home():
    answer, query = "", ""
    if request.method == "POST":
        query = request.form["query"]
        resp = requests.post(f"{API_URL}/chat",
            headers={"Authorization": f"Bearer {API_KEY}"},
            json={"notebook_id": "your-id", "message": query, "stream": False}
        )

        answer = resp.json().get("response", "Error")
        return render_template_string(HTML, query=query, answer=answer)

if __name__ == "__main__":
    app.run(port=5000)

```

n8n Automation Recipes

- **RSS → Summarize → Slack:** RSS Feed Read → POST /api/sources → Wait → POST /api/chat → Slack Message
- **Email PDF → Auto-Ingest:** Gmail Trigger (PDF attachment) → Download → POST /api/sources → Reply
- **Daily Podcast Newsletter:** Schedule (7 AM) → POST /api/podcast/generate → Wait → GET download → Email MP3

Zapier Recipes

Trigger	Action
New Google Drive file	POST /api/sources (upload)

Trigger	Action
New Airtable row	POST /api/sources (URL)
RSS new item	POST /api/sources → POST /api/chat (summarize)

Next: Chapter 8 — Monitoring, Alerting, and Maintenance → # Chapter 8: Monitoring, Alerting, and Maintenance

Estimated reading time: 15 minutes **What you'll build:** A complete monitoring stack with Prometheus + Grafana, alerting rules, log management, update strategy, and long-term maintenance checklist. **Prerequisites:** Chapters 1-4 — production deployment running.

You've deployed Open Notebook with TLS, backups, multi-user support. The final discipline: knowing when something's wrong before your users tell you.

Monitoring Stack: Prometheus + Grafana

Add a `docker-compose-monitoring.yml` with 4 services:

- **Prometheus** — metrics collection and alert evaluation
- **Grafana** — dashboards and visualization
- **Node Exporter** — host-level metrics (CPU, memory, disk)
- **cAdvisor** — container-level metrics (CPU, memory per service)

Key Alert Rules

```
groups:
  - name: open-notebook
    rules:
      - alert: OpenNotebookDown
        expr: up{job="open-notebook"} == 0
        for: 2m
        severity: critical

      - alert: DiskSpaceCritical
        expr: (node_filesystem_avail_bytes / node_filesystem_size_bytes) < 0.15
```

```

        for: 2m
        severity: critical

- alert: HighMemoryUsage
  expr: (1 - node_memory_MemAvailable_bytes/node_memory_MemTotal_bytes)
  * 100 > 90
  for: 10m
  severity: warning

- alert: GPUOutOfMemory
  expr: DCGM_FI_DEV_FB_USED / DCGM_FI_DEV_FB_TOTAL > 0.95
  for: 5m
  severity: critical

- alert: ContainerDown
  expr: time() - container_last_seen{name=~"open-notebook.*|ollama|
surrealdb"} > 60
  for: 2m
  severity: critical

- alert: BackupTooOld
  expr: time() - backup_last_success_timestamp > 90000
  for: 1m
  severity: warning

```

Grafana Dashboard — Key Panels

Panel	PromQL
Instance Up	<code>up{job="open-notebook"}</code>
Disk Free (GB)	<code>node_filesystem_avail_bytes{mountpoint="/opt"} / 1e9</code>
RAM Used %	<code>(1 - node_memory_MemAvailable/node_memory_MemTotal) * 100</code>
Container CPU	<code>sum(rate(container_cpu_usage_seconds_total{name=~"open-notebook.*"}[5m])) by (name) * 100</code>

Alert Notifications

Grafana → Alerting → Contact points → Email / Slack Webhook / Discord Webhook / Telegram Bot

Log Management

```
# docker-compose.yml - structured logging
services:
  open-notebook:
    logging:
      driver: "json-file"
      options: { max-size: "50m", max-file: "3" }
```

```
# View errors
docker compose logs open-notebook 2>&1 | grep -i "error\|exception"

# Time range
docker compose logs --since=1h open-notebook
```

For centralized log search across all services, add Loki + Promtail to the monitoring stack.

Update Strategy

```
# Pre-update: backup, read changelog
/opt/open-notebook/backup.sh
curl -s https://api.github.com/repos/lfnovo/open-notebook/releases/latest |
jq '.tag_name'

# Update
docker compose pull
docker compose up -d --force-recreate

# Verify
sleep 10
curl -sf http://localhost:5055/health && echo "[OK]" || echo "[FAIL]
FAILED"

# Rollback: pin previous version, docker compose up -d, restore backup
```

Pin versions in production: use `v1.8.3` not `:latest`.

Performance Tuning

```
# Profile slow nginx responses
tail -1000 /var/log/nginx/access.log | awk '{print $NF}' | sort -n | tail -20

# Docker resource usage
docker stats --no-stream

# SurrealDB indexes for common queries:
# DEFINE INDEX source_notebook_idx ON source FIELDS notebook;
# DEFINE INDEX note_notebook_idx ON note FIELDS notebook;
```

nginx caching for repeated API queries:

```
# Cache /api/chat responses; never cache uploads or podcast generation
proxy_cache_path /var/cache/nginx levels=1:2 keys_zone=api_cache:10m
max_size=1g;
```

Maintenance Checklist

Weekly (5 min)

```
docker compose ps && df -h /opt/open-notebook && tail -20 /var/log/nginx/error.log
```

Monthly (15 min)

```
/opt/open-notebook/backup.sh && sudo certbot renew --dry-run
docker system prune -a --volumes
/opt/open-notebook/maintenance.sh
```

Quarterly (1 hr)

- Full restore test on staging
- Review alert thresholds
- Review API costs per team
- Update pinned image tags

- Review security config

Annual

- Full DR drill (restore from off-site backup to fresh server)
 - Cost review, user survey, feature evaluation
-

End of Production Guide — Open Notebook

Full guide: 10 chapters (Ch 0-1 free preview, Ch 2-8 paid). Generated by OpenClaw, 2026-06-26.